

**µProcessador 0** Instalação dos apps para VHDL – Linux/Windows

**TL; DR:** É preciso instalar um compilador VHDL (*ghdl*) e um visualizador de formas de onda (*gtkwave*), no mínimo, e usar um bom editor de código (*Visual Studio Code*), então baixe eles, instale e teste. Há algumas alternativas, listadas numa seção.

**Índice Clicável**

LINUX

WINDOWS

ALTERNATIVAS

PLUGINS VSCODE

**L I N U X****GHDL**

Nos derivados de Debian/Ubuntu, deve estar nos repositórios, portanto é só fazer `sudo apt install ghdl`.

Temos alternativas para outras distribuições (sempre escolha a versão do mcode, que é mais leve):

- Instalar o tarball da versão 3.0 (veja <https://github.com/ghdl/ghdl/releases/tag/v3.0.0>)<sup>1</sup>;
- Usar o Docker para instalação (o pessoal da USP<sup>2</sup> tem uma referência para instalar tanto o Docker quanto os containers);
- Instalar o VHDPlus e usar o gerenciador de pacotes dele;
- Baixar os fontes da versão 1.0 e compilar (não testei).

**GTKWave**

Também deve estar nos repositórios, portanto é só fazer `sudo apt install gtwave` em *Ubuntu-like*s.

A alternativa, como última opção, é baixar o tarball de <https://sourceforge.net/projects/gtkwave/files/> e compilar para gerar o aplicativo (pode ser casca<sup>3</sup>).

**Editor de Código**

O *Microsoft VSCode* com plugins para VHDL é bom o suficiente, mas dá pra usar qualquer um que tenha suporte para autocompletar (a linguagem é muito verborrágica). Se puder usar snippets, melhor ainda – até o *gedit* é decente se configurado de acordo.

- 1 Descompacte ele com `sudo tar -C /usr/local -xzf ghdl-gha-ubuntu-20.04-mcode.tgz` ou algo parecido. Esse `sudo tar` coloca os arquivos em `/usr/local/bin`, `/usr/local/include` e `/usr/local/lib`, o que pode ser alterado à gosto do freguês.
- 2 [https://edisciplinas.usp.br/pluginfile.php/5311135/mod\\_resource/content/1/PCS3115\\_2020S1\\_Instalacao\\_GHDL\\_v1.pdf](https://edisciplinas.usp.br/pluginfile.php/5311135/mod_resource/content/1/PCS3115_2020S1_Instalacao_GHDL_v1.pdf)
- 3 Neste caso, vá no site do GTKWave e leia o manual em pdf; baixe o pacote `.tar.gz`, descompacte e leia o arquivo `README.txt`, que tem uma seção sobre instalação no Ubuntu, que pode ajudar outras distribuições também.

## W I N D O W S

É recomendável configurar o *path* de execução (o caminho default de aplicativos) para o GHDL e para o GTKWave (veja mais adiante).

### GHDL

Tem opções de instalação sobrando:

- Baixar o *ghdl-MINGW32.zip* da versão 3.0 em <https://github.com/ghdl/ghdl/releases/tag/v3.0.0> (o executável está na pasta bin);
- Utilizar o gerenciados de pacotes MSYS2 que faz tudo automaticamente;
- Usar o Docker para instalação (o pessoal da USP<sup>4</sup> tem uma referência para instalar tanto o Docker quanto os containers);
- Instalar o VHDPlus e usar o gerenciador de pacotes dele;
- Baixar os fontes da versão 1.0 e compilar (não recomendado).

### GTKWave

O mais simples é obter um binário já executável, descompactá-lo e executar na subpasta *gtkwave/bin/gtkwave.exe*. O mais recente é da versão 3.3.100 em 32 ou 64 bits, ou seja, o link *gtkwave-3.3.100-bin-win32* ou então o *gtkwave-3.3.100-bin-win64* – preste atenção nos nomes! – que estão em <https://sourceforge.net/projects/gtkwave/files/>.

Como última opção, podem ser baixados os fontes e compilados para gerar o aplicativo.

### Editor de Código

O *Microsoft VSCode* com plugins para VHDL é bom o suficiente, mas dá pra usar qualquer um que tenha suporte para autocompletar (a linguagem é muito verborrágica). O ideal é poder usar snippets de código.

Uma alternativa popular é utilizar o Notepad++.

### PATH do Windows para GHDL e GTKWave

#### O que é e pra que serve?

Ao instalar apenas descompactando ou copiando (digamos, na pasta *C:\ghdl*), você terá que executar ele em um terminal com o caminho qualificado, ou seja, *\ghdl\bin\ghdl teste.vhd* quando quiser compilar o *teste.vhd* que está na pasta atual. *Dependendo da IDE* ela deve ser configurada, mas talvez precise achar o executável sem configurações.

Para evitar isso e poder chamar o *ghdl* e o *gtkwave* de qualquer pasta, o PATH padrão deve ser alterado. Ele indica ao sistema os locais onde procurar aplicativos. Por exemplo, sem o PATH adequado você tem que executar com o caminho completo:

```
> C:\Users\Juliano\ArqComp\GTKWAVE\bin\gtkwave.exe C:\Users\Juliano\Ex\ondas.ghw
```

Ou então ir até a pasta manualmente:

```
> cd C:\Users\Juliano\ArqComp\GTKWAVE\bin
> gtwave.exe C:\Users\Juliano\Ex\ondas.ghw
```

Ou então coloque este caminho no path padrão, como feito para o *ghdl*, e execute sem o full path de qualquer lugar:

```
> cd C:\Users\Juliano\Ex
> gtwave ondas.ghw
```

### Como configurar

Siga os passos ou procure instruções na internet:

1. Vá no *Painel de Controle* e na barra de busca, digite *variáveis*.
2. Clique em *Editar as variáveis de ambiente*; se escolher para o sistema ele abre outra janela e será para todos os usuários do PC, senão é só pra você.

<sup>4</sup> [https://edisciplinas.usp.br/pluginfile.php/5311135/mod\\_resource/content/1/PCS3115\\_2020S1\\_Instalacao\\_GHDL\\_v1.pdf](https://edisciplinas.usp.br/pluginfile.php/5311135/mod_resource/content/1/PCS3115_2020S1_Instalacao_GHDL_v1.pdf)

3. Encontre a variável *Path* (o painel superior é do usuário, o inferior é do sistema):
  - Se a variável *Path* não estiver listada, clique em *Novo...*; coloque *Path* como nome e coloque as pastas do ghdl.exe e gtkwave.exe, separadas por ; (por exemplo, C:\ghdl\bin; c:\Arquivos de programas\gtkwave\bin).
  - Se a variável *Path* estiver listada, clique nela e em *Editar*. Inclua então duas novas linhas, uma para cada aplicativo (por exemplo, uma com c:\ghdl\bin e outra com C:\Arquivos de Programas\gtkwave\bin).
4. Para testar, abra um terminal (cmd, por ex.) e digite ghdl -v (que deve mostrar a versão do GHDL) e depois gtkwave (que deve abrir o aplicativo).

## ----- ALTERNATIVAS -----

Uma IDE interessante é o **VHDPPlus**, que tem um editor muito bom com integração decente do GHDL e GTKWave (além da opção de visualizador integrado), mas ele sofre da síndrome do pato (quer fazer de tudo, mas acaba ficando mais ou menos; ele até pode programar a nossa placa com FPGA).

Tem uma interface meio esquisita, comandos meio temperamentais<sup>5</sup>, sofre de pequenos bugs e tem *rolling releases* que mudam funcionalidades subitamente. Mas é bacana. Temos um [guia de instalação](#) e também a [versão da prática 1 de VHDL](#) para ele (é só clicar nos links); estão um pouco defasados mas contêm o fundamental.

As **outras IDE's** disponíveis como o Altera/Intel Quartus II ou o Xilinx ISE, são softwares pesadíssimos e muito complexos<sup>6</sup>. A sugestão oficial é evitá-las. Há outras alternativas (como usar o Logisim Evolution<sup>7</sup>, Verilog ou algum *toolchain* que mistura essas ferramentas), mas eu não avaliei elas.

A instalação, configuração e uso de algumas destas ferramentas podem ser bastante desafiadoras se não houver um tutorial adequado, mas às vezes há vantagens claras (p. ex., versões antigas do **Quartus II** têm simulação facilitada e visualização do circuito final gerado em RTL, ambos muito úteis).

## ----- PLUGINS VSCODE -----

Há mais de uma extensão VHDL, instale a mais popular, e não precisa mais nada.

Se você *desejar*, contudo, pode instalar o **GHDL Interface**, que chama o compilador e o visualizador de dentro do VSCode. É necessário alterar alguns *Settings*:

- Em *Ghdl-interface* › *Library: Library Directory* é preciso especificar uma pasta clicando em *Add item*:
  - Em Linux, pode ser qualquer pasta, senão a extensão buga (eu coloquei /tmp);
  - No Windows, tem que ser a pasta correta, que é a *lib\ghdl* dentro do ghdl. Eu baixei e descompactei o GHDL na pasta Downloads, então meu caminho ficou: C:\Users\Juliano\Downloads\MINGW32-mcode-standalone\lib\ghdl .
- Em *Ghdl-interface* › *Library: Work Library Path* é necessário preencher com file://, senão dá pau.
- Em *Ghdl-interface* › *Vhdl: VHDLStandard*, escolha 08.

5 Por exemplo, a linguagem default do programa *foi inventada por eles*, o “VHDP.” Portanto tem que tomar cuidado na criação do projeto, e a compilação e simulação acaba sendo com o botão direito do mouse...

6 Se você está acostumado a usar um deles, pode usar à vontade; caso contrário, não recomendo. Os editores integrados são, surpreendentemente, uma droga, e os simuladores nada amigáveis nas versões atuais destes programas.

7 Disponível em <https://github.com/logisim-evolution/logisim-evolution>, ele permite blocos em VHDL e pode realizar conexões e simulação visualmente.

Depois disto, teste.

Existem alguns *caveats* para esse plugin:

- Essa extensão *não reporta os erros de component not bound!* (Isto é o erro que dá quando ele não acha onde está o componente de outro arquivo VHDL). Sempre que você alterar o código e não houver alteração nas ondas, ou aparecer sinal com 'U' (*unknown*), compile na linha de comando pra verificar.
- No Windows, é necessário configurar o PATH nas variáveis de ambiente tanto para o GHDL quanto para o GTKWave para que o plugin funcione de acordo (veja seção anterior).
- O plugin está bichado no Windows (outubro/2021), mas é fácil botar pra funcionar.<sup>8</sup>

## Microsoft Visual Studio Code – Utilização

Sem a extensão *GHDL Interface*, basta usar os comandos `ghdl` e `gtkwave` num terminal, como descrito nas práticas. Caso decida usar esse plugin, estes são os passos.

1. Crie uma pasta para o seu projeto e coloque lá os arquivos [teste.vhd](#), [teste\\_tb.vhd](#) e [sinais.gtkw](#) (clique neles para baixar).
2. Abra a pasta no Code com “Open Folder.”
3. Compile (Analyze) o teste.vhd; para tanto, pode-se clicar com o botão direito no teste.vhd, seja no painel explorer ou no código aberto, e escolher “ghdl analyze file.”
4. Faça o mesmo para teste\_tb.vhd.
5. De forma similar, execute (*ghdl run unit*) o teste\_tb.vhd. Uma janela vai pedir que você dê o nome do arquivo com as ondas (.ghw), que vai aparecer na pasta. Não é preciso elaborar nada (*Elaborate*).
6. Clicando com o botão direito em cima do arquivo .ghw, você pode chamar o GTKWave.
7. No GTKWave, vá no menu *File => Read Save File* e abra o arquivo *sinais.gtkw*. Você deverá ver as formas de onda da tela mostrada no final do tutorial de instalação do VHDPlus.

Dicas:

- Perceba que o Code *não recompila automaticamente todos os arquivos!* Portanto você deve recompilar manualmente todos os arquivos que alterar, ou fazer um script pra executar na linha de comando.
- As compilações parciais são guardadas no arquivo *work-obj08.cf*, mas às vezes ele fica desatualizado. Frequentemente apague este arquivo e recompile todos os fontes novamente.
- Como já mencionado, *erros de binding não são reportados*. Caso o comportamento esteja estranho, com alterações que não produzem efeitos ou sinais marcados com valor 'U', apague o arquivo .cf e recompile tudo do zero, mas na linha de comando de um terminal, para poder verificar os erros.

8 Talvez seja uma péssima gambiarra, mas eu nunca tinha nem visto JavaScript ou usado o VSCode, então fica assim mesmo. Ressalto isso é só para o Windows, no Linux não precisa. Vá lá no arquivo `src\ghdlinterface.js` da extensão (no meu Windows, está em `C:\Users\juliano\.vscode\extensions\johannesbonk.ghdl-interface-1.1.2`). A linha 224 é:

```
const simFilePath = fileInfos.path + '.ghw';
```

Substitua ela pelas linhas:

```
const simFilePathAux = fileInfos.path;
let simFilePath = '';
if (simFilePathAux[2]===':') {
    simFilePath = simFilePathAux.substring(1); // só no Windows, buga no Linux
}
```